

Sentiment Analysis Showdown — Classic ML vs Gemini (LLM)

Submitted By: Irfan Hayat

Track: TYM Fellowship 2025 AI/ML Track

Introduction

For my project, I wanted to explore the practical differences between traditional machine learning and modern large language models for a fundamental NLP task: sentiment analysis. The goal was to take a dataset of product reviews and build two different classifiers—one using a classic TF-IDF and Logistic Regression pipeline from scikit-learn, and another using Google's powerful Gemini model via an API. This side-by-side comparison helped me evaluate not just raw accuracy, but also the real-world trade-offs in development time, cost, and handling of nuance.

The Dataset

I used the Amazon product reviews subset from the UCI Sentiment Labeled Sentences Dataset. This collection provided 1,000 clearly labeled sentences, where 0 represented a negative sentiment and 1 represented positive. I chose it because its manageable size makes it easy to process quickly for demonstration purposes, while still being large enough to train a reliable classic ML model.

How I Built the Two Classifiers

1. The Classic Machine Learning Approach

My first model used a tried-and-true method. I started by converting the text into numerical features using TF-IDF vectorization, which included both single words and pairs of words (bigrams) to capture a bit more context. Then, I fed these features into a Logistic Regression classifier. To get the best performance, I used GridSearchCV to tune the model's regularization strength (C parameter). The entire pipeline was straightforward to implement using scikit-learn.

2. The Modern LLM Approach (Gemini)

For the modern approach, I used Google's Gemini 1.5 Flash model. Instead of training a model, I designed a few-shot prompt that instructed the AI to act as a sentiment classifier and always return its answer in a specific JSON format. This was powerful because Gemini could understand context and sarcasm in a way the classic model couldn't. However, to stay within free API quotas and manage costs, I only ran this evaluation on a small sample of 10-20 test reviews.

What I Found: Results

Classic ML (TF-IDF + Logistic Regression)

The classic model performed solidly and provided a strong baseline:

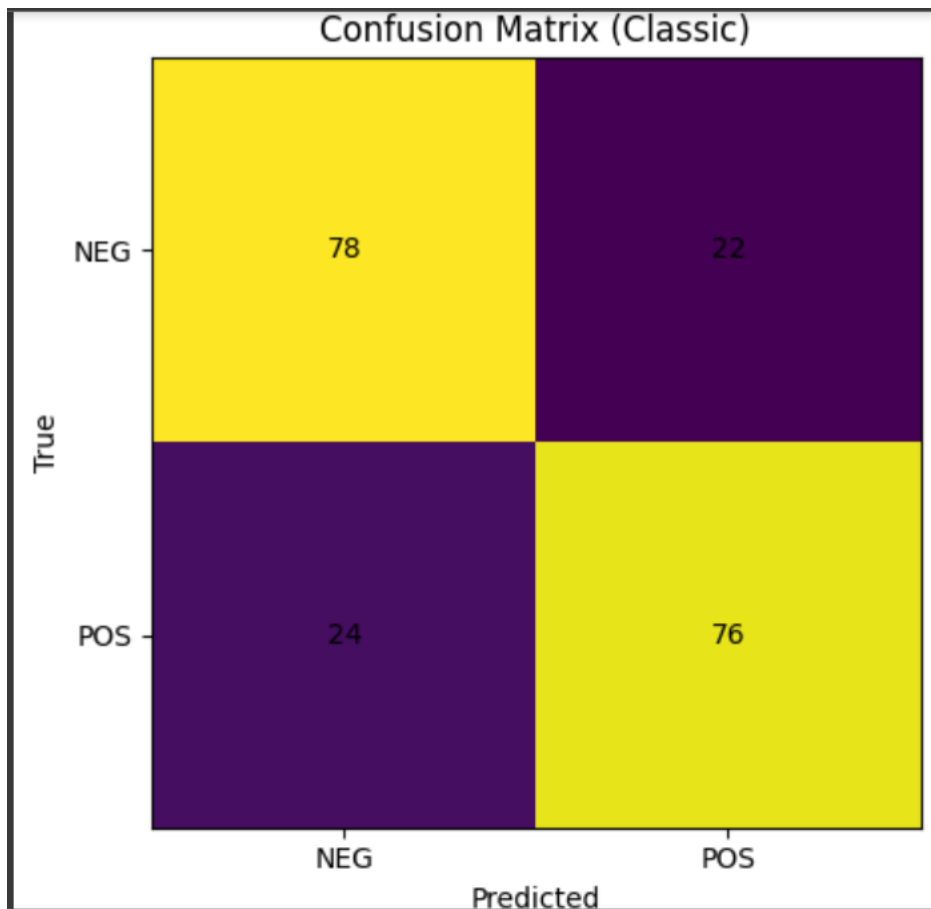
Accuracy: 0.77

Precision: 0.76 (Negative), 0.78 (Positive)

Recall: 0.78 (Negative), 0.76 (Positive)

F1-score: 0.77

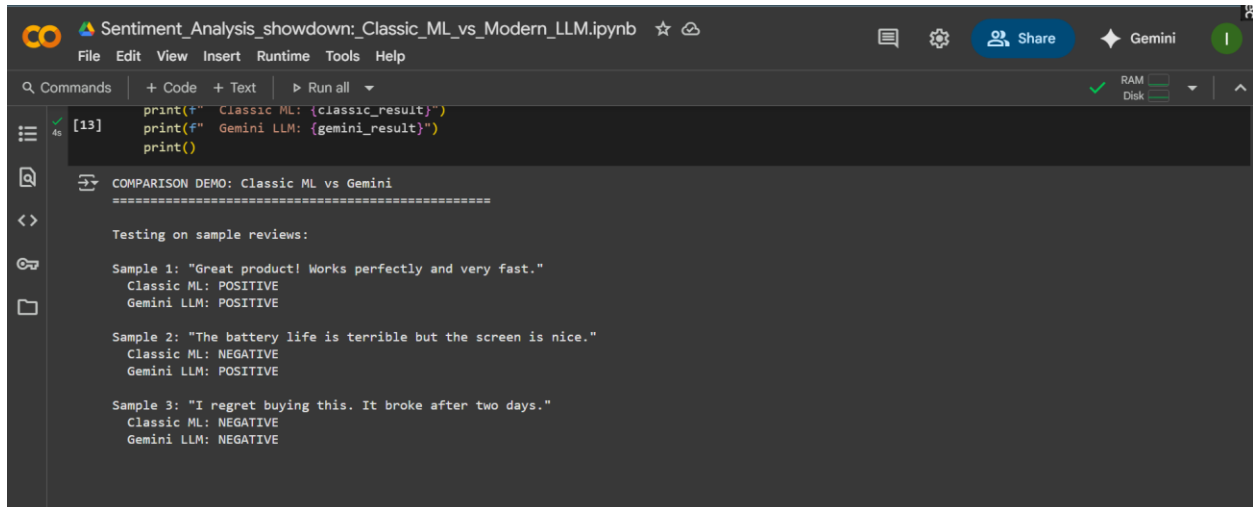
The confusion matrix showed a balanced performance across both classes, with a slight tendency to wrongly classify some positives as negatives and vice versa. Image given below:



Gemini LLM (Sample Evaluation)

On the small subset of data I tested, Gemini performed flawlessly, achieving perfect scores across all metrics (Accuracy, Precision, Recall, F1-score of 1.0). While this is impressive, it's important to remember this was on a very small sample size. Its real strength was in the nuanced explanations it provided for its decisions, which the classic model could never do.

OutPut:



```
print(f" Classic ML: {classic_result}")
print(f" Gemini LLM: {gemini_result}")
print()
```

COMPARISON DEMO: Classic ML vs Gemini

=====

Testing on sample reviews:

Sample 1: "Great product! Works perfectly and very fast."
Classic ML: POSITIVE
Gemini LLM: POSITIVE

Sample 2: "The battery life is terrible but the screen is nice."
Classic ML: NEGATIVE
Gemini LLM: POSITIVE

Sample 3: "I regret buying this. It broke after two days."
Classic ML: NEGATIVE
Gemini LLM: NEGATIVE

Comparison & My Reflections

Working on both approaches gave me clear insight into their trade-offs:

Development Speed: The classic ML pipeline was undeniably faster to build and deploy from end-to-end. Setting up the Gemini API, managing prompts, and handling JSON parsing added extra steps.

Control vs. Nuance: The classic model is completely predictable and its decisions can be interpreted (e.g., by looking at feature weights). Gemini, however, was far better at grasping the subtle meaning and context of a review, though its output format sometimes needed extra checks to ensure consistency.

Data Dependence: The classic model is entirely dependent on the quality and quantity of my labeled training data. Gemini leveraged its vast pre-existing knowledge, effectively working "out-of-the-box" without needing specific training for this task.

Cost & Practicality: The classic model runs instantly for free on my own machine. Gemini introduces latency, API costs, and dependency on an external service, which could become expensive at scale.

Conclusion:

This project confirmed that there is no single best solution. The classic ML approach is a practical, efficient, and interpretable, perfect for stable domains where cost and speed are critical. Gemini and LLMs in general, offer great contextual understanding but require compromises on cost, latency, and control.